

# Provable Discourse and Process Tracing for Agent Architectures

Geist Atlas · Module 4 · Apparatus as Method

Liam Magee\*

Education Policy, Organization and Leadership  
University of Illinois Urbana-Champaign

April 2026

## Provable Discourse and Process Tracing for Agent Architectures

[**METHODS**] Methods / apparatus contribution · **Family:** Apparatus as Method

**Claim:** The evaluation apparatus — process tracing, machine-checked claims, symmetry rules, reproducible provenance — is a transferable method for mechanistic LLM evaluation, not just background infrastructure.

The apparatus-as-method module: superego-critique taxonomy, the provable-discourse framework that verifies every mechanism claim against data, and the reproducibility chain.

*Derived view of `paper-full-2.0.md` v3.0.157 — §5.3, §5.9, §5.11, §7.4. The paper is canonical; edit it there, not here.*

### 5.3 Superego Critique Taxonomy

**Purpose** Classify what the superego actually objects to, building an empirical taxonomy from the data rather than imposing one from theory. This directly tests Mechanism 2 (error correction): if recognition changes the ego’s receptivity to critique (Section 3.2), we should see different critique patterns and revision outcomes under recognition vs. baseline conditions.

---

\*This sentence is the only one actually authored by Liam Magee in this paper.

**Data Source** Dialogue log files in `logs/tutor-dialogues/` contain the full ego→superego→ego\_\_revised chain with verbatim text. An extraction script (`scripts/extract-superego-critiques.js`) harvests all superego review entries and learner superego deliberation entries into structured JSONL format.

## Method

1. **Extraction:** Harvest all ego-superego exchanges from dialogue traces, yielding critique text, verdict (approved/rejected), confidence, intervention type, and surrounding context (ego draft, ego revision).
2. **Classification:** An LLM-based classifier (`scripts/classify-superego-critiques.js`) assigns each critique to a 10-category taxonomy:
  - Content accuracy, learner model failure, tone mismatch, structural scaffolding
  - Premature resolution, sycophancy detection, repetition/stalling, autonomy violation
  - Recognition failure (baseline-specific), redirection without engagement
3. **Quantification:** Frequency distributions by condition (recognition vs. baseline), model, and scenario. Chi-squared or Fisher’s exact test for distributional differences.

## Predictions

- Recognition-primed superegos should catch more *relational* failures (sycophancy, premature resolution, autonomy violation); baseline superegos should focus on *content/structural* issues.
- Recognition ego revisions should be more *substantive*; baseline ego revisions should be more *cosmetic* (ego\_compliance).

**Human Validation Pilot** Because the 10-category taxonomy is LLM-assigned (`scripts/classify-superego-critiques.js`, Haiku 4.5 classifier by default), we operationalise a human validation pilot rather than treating the taxonomy as self-evidently valid. The pilot apparatus lives in the repository and is runnable end-to-end by an external reviewer:

1. **Sampling** (`scripts/human-validation-sample.js`): Stratified random draw of 40 classified critiques with 4 per category across the 10 substantive labels, using a deterministic RNG seed (mulberry32, seed=20260416) for reproducibility. Categories with sparse representation in the classified corpus (CONTEXT\_BLINDNESS n=4; REGISTER\_MISMATCH n=0) are filled by deficit roll-over from larger buckets, with the gap logged for reviewer inspection. The script emits two files: a rater packet CSV (item\_id + feedback text + ego draft + ego revision + learner-context snippet, no LLM labels) and a matched key JSONL holding the LLM labels plus metadata, joined on a SHA-1 item\_id.
2. **Codebook** (`docs/research/human-coding-codebook.md`, v1.0): Per-category definitions, signal phrases, boundary rules for adjacent categories (CONTEXT\_

BLINDNESS vs FABRICATION, RECOGNITION\_FAILURE vs EMOTIONAL\_NEGLECT, RECOGNITION\_FAILURE vs LACK\_OF\_AGENCY), a 12-step decision flowchart, CSV filling instructions, and one in-corpus exemplar per category (with a constructed exemplar for REGISTER\_MISMATCH given its n=0).

3. **Analysis** (`scripts/human-validation-analyze.js`): Auto-discovers `exports/human-validation-pilot` joins each rater to the key JSONL by `item_id`, and computes (a) Cohen’s  $\kappa$  for every rater pair including each rater vs. the LLM classifier, (b) Fleiss’  $\kappa$  across the full panel where every item has complete coverage, (c) per-category F1 scores, and (d) full confusion matrices.  $\kappa$  values are interpreted using Landis-Koch bands (<.21 slight, .21-.40 fair, .41-.60 moderate, .61-.80 substantial, >.80 almost perfect). The target floor for publishable reliability is  $\geq 0.60$  (**substantial agreement**).
4. **Inter-LLM baseline**: Re-classifying the same 40-item sample with a second LLM (e.g., Sonnet 4.6) and passing it as a synthetic rater (`--synthetic-rater`) yields an LLM-LLM lower-bound reference. Human-LLM below this baseline would indicate the human coding is less reliable than disagreement between two well-behaved LLMs; human-LLM substantially above it indicates the human panel is not just tracking LLM idiosyncrasies.

The pilot scales ( $k \geq 2$  raters,  $n=40$  items, 10 categories) follow Cohen (1960) and Hallgren (2012) for stratified categorical reliability with minimal items-per-cell to yield interpretable per-category F1. A second phase scaling to  $n=100$ -150 items is budgeted for the final paper revision if the pilot meets the floor. Pilot  $\kappa$  values and any category with  $F1 < 0.60$  will be flagged as “pilot-unstable” and treated as a validity caveat in §5.3 rather than a taxonomy redesign trigger.

**Inter-LLM Baseline Result (v3.0.32, preliminary)** The inter-LLM baseline run is complete. Haiku 4.5 (the original classifier,  $N=500$  corpus) and Sonnet 4.5 (rerun on the same 40-item sample with identical taxonomy prompt, `temperature=0`, `response_format=json_object`) yield **Cohen’s  $\kappa = 0.633$  (substantial agreement per Landis-Koch),  $n=40$ , observed agreement 0.675, chance agreement 0.114**. Per-category F1 partitions into three tiers: *high* ( $F1 \geq 0.75$ ) — LACK\_OF\_AGENCY (1.000), CONTEXT\_BLINDNESS (0.889), MEMORY\_FAILURE (0.833), VAGUENESS (0.750); *moderate* ( $F1$  0.50–0.75) — RECOGNITION\_FAILURE (0.625), REDIRECTION (0.571), EMOTIONAL\_NEGLECT (0.500), PEDAGOGICAL\_MISJUDGMENT (0.500); *pilot-unstable* — FABRICATION ( $F1$  undefined: Haiku assigned 4 items, Sonnet assigned 0, distributing them across CONTEXT\_BLINDNESS/VAGUENESS/MEMORY\_FAILURE). The confusion-matrix asymmetry points to RECOGNITION\_FAILURE as a catch-all under Sonnet’s decision policy: Haiku’s EMOTIONAL\_NEGLECT, PEDAGOGICAL\_MISJUDGMENT, and some REDIRECTION items all collapse to RECOGNITION\_FAILURE under Sonnet. This matches the codebook’s boundary-rule guidance that RECOGNITION\_FAILURE is the broadest diagnostic frame and should be applied only when the critique explicitly invokes intellectual autonomy as the core problem. The human-rater pilot will use this  $\approx 0.63$  as its lower-bound reference: human-LLM substantially above 0.63 indicates the human panel is not merely tracking LLM idiosyncrasies; near or below 0.63 indicates the taxonomy may not generalize beyond LLM-LLM reliability. FABRICATION will require explicit codebook rein-

forcement (e.g., elevated to decision-flowchart step 2) or retirement if human raters likewise fail to converge on it.

## 5.9 Provable Discourse Framework

**Architecture** The provable discourse framework treats every quantitative claim in the paper as an executable test. A claim is a structured YAML entry with five components:

1. **Statement:** A regex pattern that locates the claim in the paper text, with a minimum occurrence count. If the pattern is not found, the claim is flagged as *orphaned*—the paper no longer contains the text it is supposed to verify.
2. **Evidence:** An executable adapter that extracts the actual value from data (database, dialogue logs, manifest files, or source code). Each adapter type encapsulates a specific extraction pattern (see taxonomy below).
3. **Assertion:** A machine-checkable predicate (equality, approximation with tolerance, inequality, existence) applied to the extracted value.
4. **Dependencies:** Optional upstream claims that must pass before this claim is evaluated. If a dependency fails, the claim cascades to *blocked* status without evaluation.
5. **Remediation:** Concrete steps to fix a failing claim, ensuring that failures are actionable rather than merely diagnostic.

The framework is implemented in `services/provableDiscourse.js` (2,700 lines) with claims distributed across four YAML files: a root configuration (`config/provable-discourse.yaml`), generated claims from Paper 1.0, manually authored claims, and mechanism-specific claims for Paper 2.0. The current ledger contains **119 claims** across **18 evidence adapter types**, validated by **6 symmetry rules**.

**Evidence Adapter Taxonomy** Each adapter type encapsulates a specific data extraction pattern. The 18 types fall into five functional categories:

**Data integrity** (verify that data exists and is consistent):

| Adapter                             | What It Extracts                                  | Example                  |
|-------------------------------------|---------------------------------------------------|--------------------------|
| <code>manifest_total</code>         | Total count from <code>paper-manifest.json</code> | N=4,312 primary scored   |
| <code>manifest_section_total</code> | Section-specific count from manifest              | Multi-model probe N=655  |
| <code>db_count</code>               | Row count from evaluation database with filters   | N=350 modulation dataset |
| <code>provenance_check</code>       | Hash match rate between DB and dialogue logs      | ≥95% content hash match  |
| <code>log_trace_coverage</code>     | Fraction of scored rows with available log files  | ≥95% trace coverage      |

**Effect estimation** (compute statistical quantities from data):

| Adapter                                | What It Extracts                                     | Example                   |
|----------------------------------------|------------------------------------------------------|---------------------------|
| <code>effect_size</code>               | Cohen’s d between groups on a scored metric          | Architecture d=0.05       |
| <code>profile_group_effect_size</code> | Cohen’s d comparing named profile groups             | Memory isolation d=1.71   |
| <code>anova_2x2</code>                 | F-statistic and $\eta^2$ from $2 \times 2$ factorial | Interaction F=0.26, p>.05 |
| <code>judge_pair_correlation</code>    | Pearson r between two judges’ scores                 | Cross-judge r=0.55        |

**Mechanism-specific** (added for Paper 2.0):

| Adapter                                | What It Extracts                                              | Example                             |
|----------------------------------------|---------------------------------------------------------------|-------------------------------------|
| <code>dimension_variance</code>        | Cross-dimension variance reduction by condition               | Calibration $d \leq -0.3$           |
| <code>dimension_cluster_effect</code>  | Effect size for dimension clusters                            | Recognition cluster d $\geq 0.50$   |
| <code>jsonl_critique_stats</code>      | Superego critique category frequencies from classified corpus | RECOGNITION_FAILURE $\geq 15\%$     |
| <code>trajectory_slope</code>          | Per-turn OLS regression coefficients                          | Learner slope $\beta \geq 0$        |
| <code>conditional_delta</code>         | Score change after specific learner events                    | Post-confusion adaptation $\geq -1$ |
| <code>rubric_version_comparison</code> | Cross-version score correlation                               | $r \geq -0.1$ (stability)           |

**Structural** (verify code paths and cross-references):

| Adapter                      | What It Extracts                                | Example                            |
|------------------------------|-------------------------------------------------|------------------------------------|
| <code>code_path</code>       | Grep count for patterns in source files         | $\geq 5$ matches for trace logging |
| <code>cross_reference</code> | Existence check for a referenced upstream claim | Pilot d=1.11 claim exists          |

**Theoretical** (register non-machine-checkable claims for traceability):

| Adapter                  | What It Extracts                                  | Example                                            |
|--------------------------|---------------------------------------------------|----------------------------------------------------|
| <code>theoretical</code> | Presence of related provable claims in the ledger | Three mechanisms each have $\geq 1$ provable claim |

**Worked Example: Tracing a Calibration Claim** To illustrate the full claim→evidence→source chain, consider claim `paper2.calibration.variance_reduction`:

```

- id: paper2.calibration.variance_reduction
  description: "Recognition narrows tutor output distribution."
  statement:
    pattern: "recognition narrows.*output distribution"
    flags: "i"
    min_occurrences: 0
  evidence:
    type: dimension_variance
    group_by: recognition
    output: cohens_d
    filters:
      not_null: [tutor_first_turn_score]
      like: { judge_model: "%claude%" }
  assertion:
    op: lte
    expected: -0.3
  remediation:
    - "If variance reduction d drifts above -0.3,
      update calibration claims or re-scope runs."

```

**Validation trace.** When the harness evaluates this claim:

1. **Statement check:** The regex `recognition narrows.*output distribution` is matched against the concatenated paper text. If not found, the claim is flagged as orphaned.
2. **Evidence extraction:** The `dimension_variance` adapter queries the evaluation database for all v2.2 rows with non-null tutor scores and a Claude-family judge. For each row, it computes the standard deviation across the 8 tutor rubric dimensions (from the `scores_with_reasoning` JSON column). It then computes Cohen’s *d* between the recognition and baseline groups’ dimension SDs.
3. **Assertion:** The extracted *d* is checked against `lte -0.3`. Current value:  $d = -0.52$  to  $-0.64$  across models. **Pass.**
4. **Staleness:** A SHA-256 fingerprint of the query results is compared against the stored snapshot. If the data has changed since the last validation, the claim is flagged as *stale* even if still passing—alerting the author that the underlying evidence has shifted.

This chain ensures that the paper’s calibration claim (“recognition narrows the output distribution”) is continuously verified against the actual dimension variance in the database. If a data correction, new run, or rubric change altered the variance pattern, the claim would fail or flag as stale, and the remediation steps would guide the fix.

**Dependency Graph** Claims form a directed acyclic graph (DAG) via two edge types: explicit `depends_on` declarations and implicit edges from `cross_reference` evidence (where a claim cites another claim’s result). The validation pipeline topologically sorts claims (Kahn’s algorithm) before evaluation, so upstream claims are always resolved before their dependents. If a dependency fails, all downstream claims are automatically marked *blocked* (status: warn) with a `blocked_by` annotation—they are not evaluated, since their evidence preconditions are not met. This cascading validation prevents a common failure mode in claim ledgers: a root statistic changes, but downstream claims that cite it continue to pass because they only

check for the citation’s *existence*, not its *validity*.

For example, the pilot factorial effect size (`d=1.11`, claim `paper2.s1.pilot.factorial_d`) depends on the `N=350` dataset claim (`paper.modulation.dataset_n350`). Five further claims—model-dependent architecture reversal, strategy shift frequency, baseline stalling rate, cross-judge correlation range, and the learner paradox effect size—in turn depend on the factorial claim. If the `N=350` dataset claim were to fail (e.g., because a data correction changed the sample size), the factorial claim would be blocked, and all five downstream claims would cascade to blocked status without evaluation. The `--graph` flag outputs the full DAG in Graphviz DOT format for visual inspection.

**Symmetry Rules and Consistency Checks** Beyond individual claims, symmetry rules enforce cross-claim consistency. Six rules currently operate:

1. **Paired presence:** If a tutor-side effect is claimed, the corresponding learner-side effect must also be explicitly stated (or explicitly omitted with justification).
2. **Magnitude bounds:** Paired claims (e.g., tutor architecture `d` and learner architecture `d`) must both fall within a threshold (e.g., both  $|d| \leq 0.2$  for “near-zero” claims).
3. **Material gap:** Asymmetric claims (e.g., tutor recognition  $d = -1.00$  vs. learner  $d = 0.32$ ) must exceed a minimum gap threshold to justify asymmetry framing.
4. **Mechanism consistency:** If a mechanism is claimed for recognition, the *anti-pattern* must be documented for baseline (paired evidence).
5. **Model qualification:** Any mechanistic claim must state which models it has been tested on, with replication status for each.
6. **Inventory coverage:** Every major quantitative claim in the paper text (identified by automated scanning for `N=` and statistical patterns) must map to at least one claim in the ledger. Unmapped claims are flagged as unverified.

---

## 5.11 Reproducibility Infrastructure

**Evaluation Commands** All evaluations are reproducible via the CLI:

```
node scripts/eval-cli.js run --profiles <cells> --runs N --cluster multi-turn
node scripts/eval-cli.js evaluate <runId>
```

Appendix B provides the exact commands for each evaluation run referenced in the paper.

**Provenance Chain** Every new evaluation row records: - `dialogue_content_hash`: SHA-256 of the dialogue log file - `config_hash`: SHA-256 of the cell configuration - Rubric version, judge model, and all hyperparameters

**Test Suite as Analytical Provenance** The test suite (32+ files, ~12K lines) covers not only the evaluation infrastructure but also the analytical pipeline. Tests verify that scoring, ANOVA computation, trajectory extraction, and within-test change analysis produce correct outputs on known inputs.

This makes the analysis *reproducible in a testable sense*: the code’s behavior on known inputs is verified by tests, and the exact code version is recorded with each evaluation run via `git_commit` in the `evaluation_runs` table.

## 7.4 The Apparatus as Method

This paper’s most distinctive contribution may not be any single finding about recognition or multiagent architecture. It is that the evaluation apparatus itself—the provable discourse framework, the rubric iterations, the bug corrections, the test suite—constitutes a **transferable methodology** for mechanistic LLM evaluation. Most LLM evaluation papers treat infrastructure as invisible scaffolding, reporting what was measured rather than how measurement was built, corrected, and validated. We foreground the apparatus because the process of building it mirrors the mechanisms it studies.

**7.4.1 Error Correction in Architecture and Research** The parallel between the tutoring architecture and the research process is structural, not metaphorical:

| Architecture Layer                                  | Research Layer                                              |
|-----------------------------------------------------|-------------------------------------------------------------|
| Ego generates initial response                      | Researcher generates initial claim                          |
| Superego critiques against pedagogical principles   | Provable discourse tests against evidence                   |
| Ego revises (substantive or cosmetic)               | Researcher revises (correction or rationalization)          |
| Ego compliance: revision without substance          | P-hacking: evidence without substance                       |
| Recognition gives ego capacity for genuine revision | Provable discourse infrastructure forces genuine correction |
| Final output scored by external judge               | Final paper evaluated by external reviewers                 |

The companion pilot study documented nine post-extraction corrections across five development phases, each following the superego-intervention pattern: a validation mechanism catches a discrepancy, and the researcher must choose between genuine revision and cosmetic compliance. The provable discourse framework constrains this choice toward genuine revision by making the claim-evidence relationship machine-verifiable. Characteristic examples include the February 6 active-control reframing (a model confound caught by the data led to revising “placebo proves recognition is the only active ingredient” into “active control scores between base and recognition”), the February 16 judge-version unification (a version audit caught a confound that shifted  $d$  from 0.80 to 1.11 after cascade re-scoring), and the February 20 learner-prompt-leakage fix (a code review forced clean re-generation of all dynamic-learner data). Each correction made the findings *less clean* and *more accurate*.

**7.4.2 The Provable Discourse Framework** The framework treats paper claims as executable tests: every quantitative claim has a *statement pattern*, machine-executable *evidence*, an *assertion* predicate, *dependencies* on upstream claims, and *remediation steps*. 119 claims



currently run against 18 evidence-adaptor types organised as a dependency DAG with topological validation; §5.9 documents the full adaptor taxonomy and a worked claim trace.

The dependency graph is the framework’s key structural contribution. When a root claim fails, dependents cascade to *blocked* status without evaluation, preventing false confidence from stale downstream claims. During Paper 2.0 development a data correction changed the learner superego paradox effect size from  $d = 1.43$  to  $d = 3.05$  (§6.5); the graph propagated this to five downstream claims that had cited the old value, each flagged stale until updated. Without the graph, those five claims would have continued to pass while quietly citing an obsolete number—exactly the silent inconsistency that erodes trust in complex empirical papers. The YAML claim format requires no programming and the 18 evidence adaptors cover all 119 claims, so the cost of adoption is low.

**7.4.3 The Rubric Iteration as Construct Refinement** The rubric evolved through four versions ( $v1.0 \rightarrow v2.0 \rightarrow v2.1 \rightarrow v2.2$ , detailed in §5.2.6), each responding to specific measurement problems:  $v1.0$  ceiling effects,  $v2.0$  judge-input confounds,  $v2.1$  dimension-redundancy findings,  $v2.2$  literature-informed  $14 \rightarrow 8$  consolidation. Each iteration follows the error-correction pattern: an initial rubric (ego) produces scores that are challenged by measurement problems (superego), and the revised rubric is a genuine reconceptualisation (strategic revision) rather than a cosmetic fix. The  $14 \rightarrow 8$  dimension reduction is itself a finding about what the evaluation *actually* measures — and §8.6’s PCA on the  $v2.2$  scores extends that finding further: the 8 dimensions still load on one factor, suggesting the apparatus is a construct-development process as much as a measurement tool.

**7.4.4 The Test Suite as Analytical Provenance** The test suite (54 files across `tests/` and `services/__tests__/`) covers not just system functionality but analytical correctness: provenance tests (44/44 passing) verify every scored row has a traceable path from cell config through dialogue execution to judge evaluation; superego guard tests (111 tests) verify single-agent cells cannot produce superego traces; ANOVA tests verify statistical computations match reference implementations. The suite makes the analysis *reproducible in a testable sense* — not just “you can run our code” but “our code produces expected output on known inputs, and here are the tests that prove it.”

---

**Neighbouring modules:** Module 8 — Rubrics as Instruments and Optimization Signals.